Студент: Коробов Евгений, 324

Отчет о выполнении задания по параллельному программированию. Вариант 413

1) В архиве лежат следующие программы:

- var413.c – текст исходной программы
- var413_op.c – текст оптимизированной программы
- var413_for.c – текст программы, использующей технологию OpenMP #pragma omp for
- var413_task.c – текст программы, использующей технологию OpenMP #task
- var413_MPI.c – текст программы, использующей технологию MPI

2) Оптимизация исходной программы:

a) Поменяем порядок переменных в циклах (с for j … for i … на с for i … for j …). Это обеспечит оптимизацию доступа к памяти. Также сделаем переменные цикла локальными для каждой функции.

b) Массив A будем передавать явно в функцию (для локального кеширования данных)

1. Времена (в сек.) работы исходной и оптимизированной программ

Программа \ N	160	480	1440	4800
Исходная	0,073396	0,923563	7,891208	130,6132
Оптимизированная	0,051372	0,615101	5,214922	55,65104

3) Распараллеливание программы с помощью технологии OpenMP #pragma omp for:

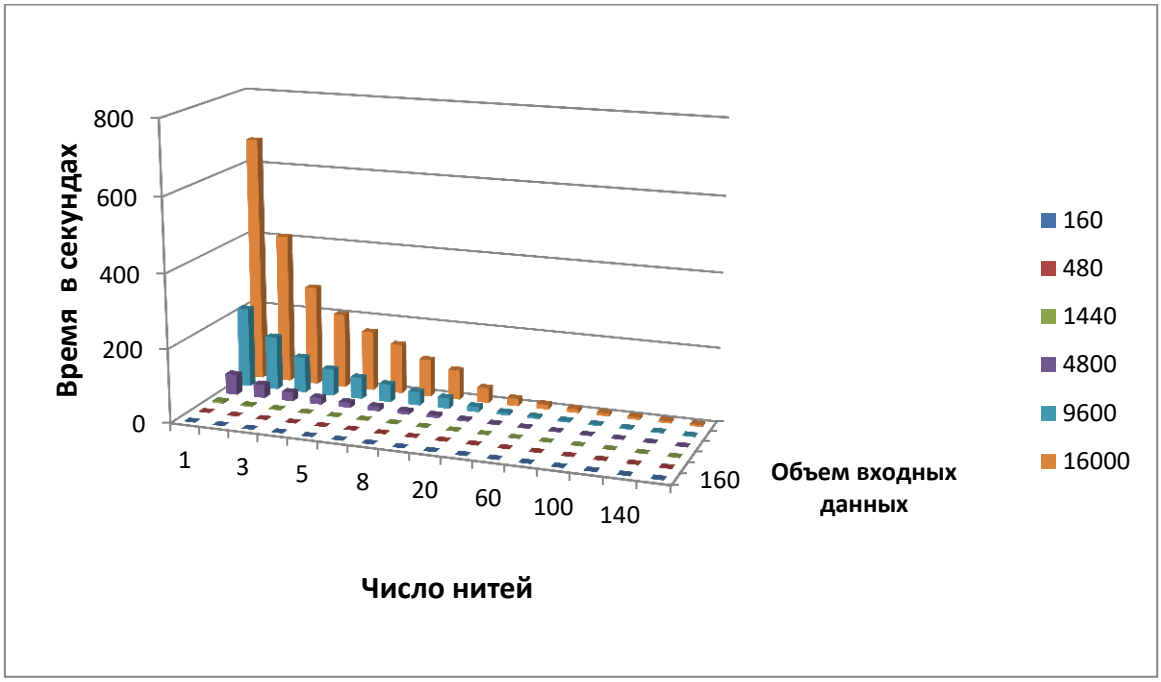
Применим технологию OpenMP #pragma omp parallel for для функций relax(), init(), verify(). Будем использовать приватную переменную j. Для relax() для вычисления максимальной епсилон используем редукцию max.

Измерим время выполнения программы с различным числом нитей и размером данных:

Нити\N	160	480	1440	4800	9600	16000
1	0,067	0,593	5,128	56,342	219,468	671,986
2	0,04	0,368	3,313	36,818	147,222	409,304
3	0,027	0,245	2,208	24,543	98,154	272,676
4	0,025	0,184	1,656	18,412	73,612	204,514
5	0,014	0,147	1,325	14,729	58,898	163,624
6	0,013	0,123	1,104	12,278	49,076	136,339
8	0,012	0,092	0,828	9,264	36,843	102,257

10	0,008	0,074	0,662	7,363	29,449	81,835
20	0,004	0,037	0,331	3,686	14,728	40,923
40	0,002	0,018	0,166	1,845	7,364	20,458
60	0,0014	0,012	0,111	1,231	4,916	13,636
80	0,001	0,009	0,083	0,922	3,681	10,234
100	0,0008	0,007	0,066	0,749	2,944	8,182
120	0,0007	0,006	0,055	0,616	2,457	6,825
140	0,0006	0,005	0,047	0,537	2,194	5,843
160	0,0005	0,005	0,041	0,462	1,849	5,119

2. Времена (в сек.) работы программы при различном числе нитей и объеме данных



3. График зависимости времени работы for-программы от числа нитей и объема данных

3) Распараллеливание программы с помощью технологии OpenMP #task:

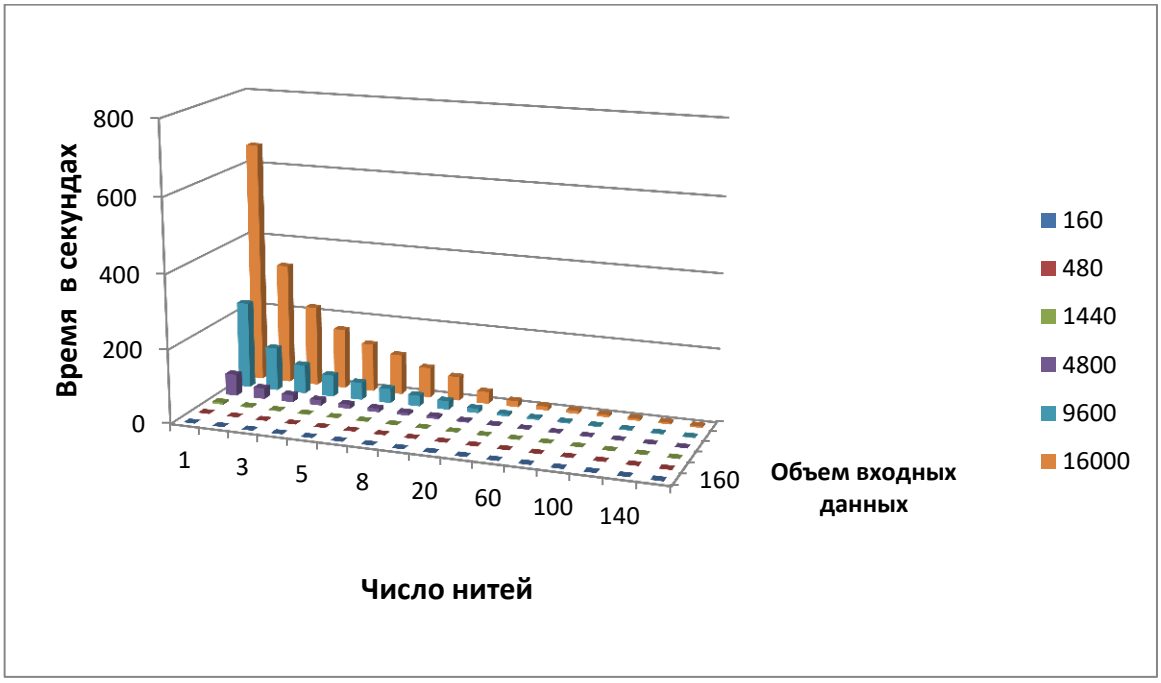
Создадим параллельные области для выполнения 1 потока в функциях relax(), init(), verify() с помощью #pragma omp parallel и #pragma omp single. Применим технологию #pragma omp task для распределения вычислений между задачами. В функции relax() объявим критическую зону #pragma omp critical, где выбирается максимальная епсилон.

Измерим время выполнения программы с различным числом нитей и размером данных:

Нити\N	160	480	1440	4800	9600	16000
1	0,065	0,592	5,328	59,267	236,824	657,778
2	0,032	0,296	2,664	29,663	118,450	328,889

3	0,021	0,197	1,776	19,733	78,933	219,259
4	0,016	0,148	1,332	14,815	59,270	164,444
5	0,013	0,118	1,066	11,846	47,365	131,556
6	0,011	0,099	0,888	9,867	39,467	109,63
8	0,008	0,074	0,666	7,443	29,607	82,222
10	0,006	0,059	0,533	5,928	23,689	65,778
20	0,003	0,031	0,266	2,960	11,848	32,889
40	0,0016	0,015	0,133	1,485	5,926	16,444
60	0,0011	0,010	0,089	0,987	3,947	10,963
80	0,0008	0,007	0,067	0,741	2,965	8,222
100	0,0007	0,006	0,053	0,592	2,368	6,578
120	0,0005	0,005	0,044	0,493	1,973	5,481
140	0,0005	0,004	0,038	0,423	1,691	4,698
160	0,0004	0,004	0,033	0,37	1,48	4,111

4. Времена (в сек.) работы #task программы при различном числе нитей и объеме данных



5.График зависимости времени работы task-программы от числа нитей и объема данных

4) Распараллеливание программы с помощью технологии MPI:

В main() массив A распределяется между процессами, матрица разбивается по, каждый процесс обрабатывает свой горизонтальный блок.

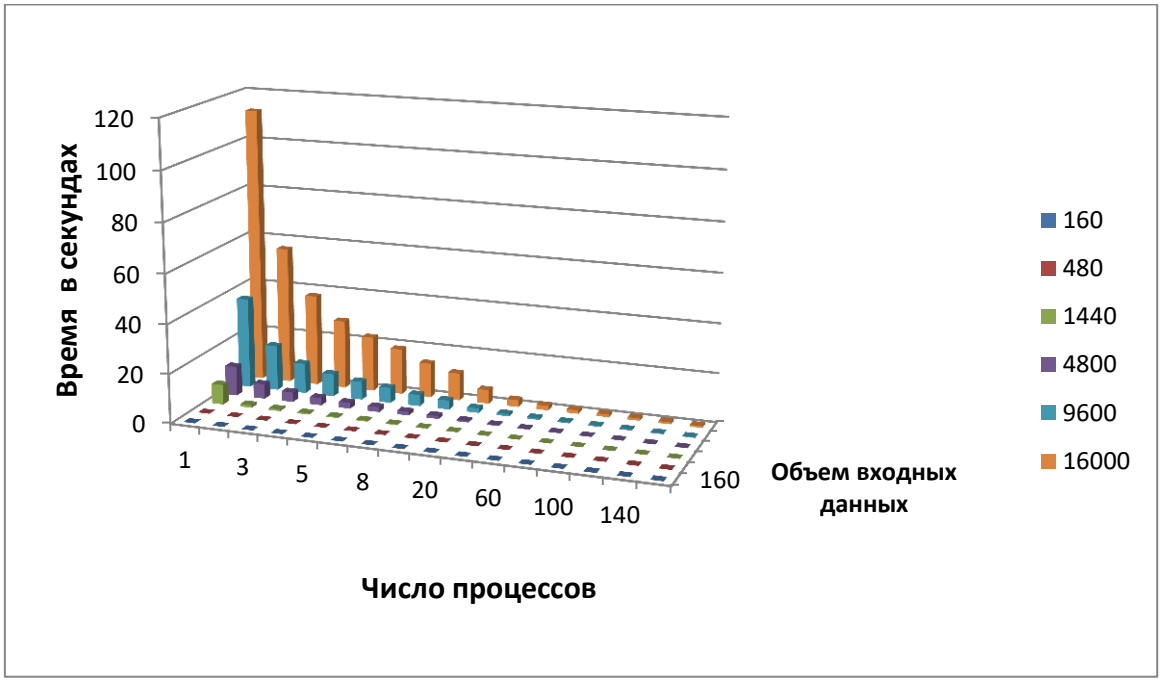
Далее каждый процесс обрабатывает свой блок, соседние по блокам процессы обмениваются значениями на граничных строках с помощью функции dop_rows() и 2 дополнительных строк (с помощью MPI_Sendrecv).

В функции relax() также используется MPI_Allreduce для определения глобального максимума Eps, а для функции verify — редукция с суммированием на нулевом процессе (MPI_Reduce).

Измерим время выполнения программы с различным числом нитей и размером данных:

6. Времена (в сек.) работы MPI программы при различном числе процессов и объеме данных

Процессы\N	160	480	1440	4800	9600	16000
1	0,066753	0,085775	8,121519	12,07398	37,09641	112,3813
2	0,033377	0,035387	1,061903	6,036989	18,54821	56,19063
3	0,022251	0,023592	0,707502	4,024659	12,36547	37,46042
4	0,016689	0,017694	0,531207	3,018494	9,274101	28,09532
5	0,01335	0,014156	0,42522	2,414796	7,419281	22,47626
6	0,011126	0,011796	0,354963	2,01233	6,185735	18,73022
8	0,008345	0,008847	0,266507	1,509248	4,637051	14,04766
10	0,006675	0,007077	0,211871	1,207398	3,709641	11,23813
20	0,003338	0,003539	0,107052	0,603699	1,854821	5,619063
40	0,00167	0,00177	0,052869	0,30185	0,92741	2,809532
60	0,001113	0,001179	0,037428	0,201233	0,618273	1,873022
80	0,000834	0,000885	0,027387	0,150926	0,463706	1,404767
100	0,000668	0,000708	0,021521	0,12074	0,370964	1,123812
120	0,000557	0,00059	0,018293	0,100617	0,309137	0,936512
140	0,000477	0,000506	0,018911	0,086243	0,264974	0,802533
160	0,000417	0,000443	0,014271	0,075464	0,231852	0,702383



7.График зависимости времени работы MPI-программы от числа процессов и объема данных

5) Выводы:

Максимальные ускорения параллельных версий относительно оптимизированной последовательной программы для максимального размера задачи (N=16000) составляют:

OpenMP-for: ~10.9 раз (при использовании 160 нитей).

OpenMP-task: ~13.5 раз (при использовании 160 нитей).

MPI: ~70 раз (при использовании 160 процессов).

OpenMP-for и OpenMP-task – сопоставимы, MPI – справляется куда лучше.